

# 受講者の習熟度が揃わない問題、AIで解決する——習熟度別サポート資料・追加課題プロンプト完全集

「この人、明らかに追いつけていない」「この人、退屈そうにしている」

研修中に両方を同時に感じながら、どちらにも満足に対応できなかった経験、ありませんか？

習熟度の差は、IT研修で必ず発生します。前提知識の差、業務経験の差、学習速度の差——理由はさまざまですが、「全員に同じ資料・同じ課題」では誰かが必ず置き去りになります。

本記事は、習熟度別のサポート資料と追加課題をAIで即席生成するプロンプト集です。ChatGPT / Claude にコピペして使えます。

---

## 目次

1. [なぜ習熟度別対応が難しいのか\(問題の整理\)](#)
  2. [プロンプト設計の基本思想](#)
  3. [【レベル下向け】フォロー資料生成プロンプト](#)
  4. [【レベル上向け】発展課題生成プロンプト](#)
  5. [【両方同時生成】差分パッケージプロンプト](#)
  6. [シーン別コピペプロンプト集](#)
  7. [生成物の品質を上げる追加指示集](#)
  8. [PowerPoint用スライドアウトラインプロンプト](#)
  9. [まとめ](#)
- 

## 1. なぜ習熟度別対応が難しいのか

IT研修の現場でよく起きる「習熟度ミスマッチ」の構造を整理します。

| 状況  | レベル下の受講者     | レベル上の受講者     |
|-----|--------------|--------------|
| 演習中 | 手が止まる・質問できない | 早く終わって手持ち無沙汰 |
| 講義中 | 用語でつまづく      | 知ってる内容を聞かされる |

| 状況      | レベル下の受講者   | レベル上の受講者    |
|---------|------------|-------------|
| グループワーク | 貢献できず萎縮    | 引っ張る役になり疲弊  |
| 研修後     | 理解不足のまま現場へ | 物足りなさ・不満が残る |

従来の対応策：

- フォロー資料を事前に準備（→ 時間がない）
- 個別に口頭説明（→ 他の受講者が待つ）
- 発展課題を別途用意（→ これも時間がない）

AIを使えば、これらを「研修直前・研修中」に即生成できます。

## 2. プロンプト設計の基本思想

すべてのプロンプトに共通する設計原則です。

- ① 研修テーマを具体的に渡す
- ② 対象者の現在地（スタートライン）を定義する
- ③ ゴール（到達点）を明示する
- ④ 出力フォーマットを指定する
- ⑤ 使用場面（演習中 / 研修後 / 事前配布）を伝える

この5点を入れるだけで、汎用的な答えではなく「使える素材」が返ってきます。

## 3. 【レベル下向け】フォロー資料生成プロンプト

### 3-1. 基本テンプレート

あなたはIT研修の教材設計の専門家です。

## 研修情報

- テーマ:【例:Gitの基本操作(add / commit / push)】

- 対象者:IT未経験～1年目のエンジニア

- 本日の研修で扱った内容:

- ローカルリポジトリの作成

- ステージングとコミット

- リモートへのプッシュ

- 使用環境:Windows / VSCode / GitHub

## ## 現状の課題

演習で手が止まっている受講者が複数います。

「コマンドの意味がわからない」「エラーが出たらどうすればいいか不安」という声があります。

## ## 依頼内容

以下の構成でフォロー資料を作ってください。

1. 「そもそも何をやっているのか」のたとえ話(日常生活の例え)
2. 今日扱ったコマンドの一覧表(コマンド・読み方・意味・具体例)
3. よくあるエラーとその対処法(3～5個)
4. 「ここまでできたらOK」チェックリスト(5項目)
5. もう一度やってみる練習課題(最小構成・手順付き)

## ## 出力形式

Markdown形式。見出し・表・コードブロックを使うこと。

コマンドはすべてコードブロックに入れること。

専門用語を使う場合は必ず括弧内に説明を添えること。

---

### 3-2. 「用語が多すぎてついていけない」向け

あなたはやさしいIT用語解説の専門家です。

#### ## 状況

研修テーマ:【例: Dockerによるコンテナ環境の構築】

問題: 受講者の一部が「仮想化」「イメージ」「コンテナ」「ボリューム」などの

用語で詰まっており、演習に進めていません。

#### ## 依頼

以下を作成してください。

##### #### A. 用語マップ(理解の地図)

今日登場した用語を関係図にしてください。

「○○は××の中に入っている」という入れ子関係を

テキストの箇条書きとインデントで表現してください。

##### #### B. 日常語への翻訳表

| Docker用語 | 日常のたとえ | 一言説明 |

の形式で10語を説明してください。

##### #### C. 「なぜこれが必要か」の理由

エンジニアの現場でDockerが使われる理由を、

「Dockerがない世界で困ること」から説明してください(3~5個の困りごと)。

出力形式: Markdown。わかりやすさ最優先。

---

### 3-3. 演習中に詰まっている人への「ヒントカード」

以下の演習課題について、4段階のヒントカードを作ってください。

### ## 演習課題

【例: Pythonでユーザーから入力を受け取り、偶数か奇数かを判定して出力するプログラムを書く】

### ## ヒントカードの仕様

「ヒント1」を見ても解けない場合だけ「ヒント2」を見る、という段階式にする。

- ヒント1: 「どこから考え始めるか」だけ教える(コードなし)
- ヒント2: 必要な命令・関数の名前だけ教える(使い方は書かない)
- ヒント3: コードの骨格(穴埋め式)を示す
- ヒント4: 完成形のコードと解説

### ## 補足

各ヒントの末尾に「これだけ見てもう一度試してみよう」という一言を添えること。

出力形式: Markdown。ヒントごとに区切り線を入れること。

---

## 3-4. 研修後の「自習ロードマップ」

あなたはIT学習コーチです。

### ## 状況

研修テーマ: 【例: SQLの基礎(SELECT / WHERE / JOIN)】

対象者: 今日の研修を受けたが、十分に理解しきれなかった受講者

目標: 2週間後に業務でSQLを使えるようにする

### ## 依頼

以下を作成してください。

1. 「今日わかっていないと困ること」の整理(優先度付き)

2. 2週間の自習スケジュール(1日15～30分想定)
3. 無料で使える練習リソース(サービス名とURLは仮でよい、用途説明を詳しく)
4. 「これができたら次へ」の自己チェックポイント(週ごとに設定)
5. よくある挫折パターンと乗り越え方

出力形式: Markdown。実践的で励ましのトーンで。

---

## 4. 【レベル上向け】発展課題生成プロンプト

### 4-1. 基本テンプレート

あなたはIT研修の上級者向け課題設計の専門家です。

#### ## 研修情報

- テーマ:【例:Gitの基本操作(add / commit / push)】
- 基本課題:すでに完了している
- 対象者:業務でGitを使った経験がある。基本操作は問題ない。

#### ## 依頼

「時間が余った上級者が、基本研修の発展として取り組める課題」を3つ作成してください。

#### ## 課題の条件

- 基本研修の内容と地続きであること(突然新しいトピックにならないこと)
- 1課題あたり20～40分で完結すること
- 業務で実際に役立つ内容であること
- 各課題に「なぜこれが業務で重要か」の説明を添えること

#### ## 出力形式

各課題について:

- 課題タイトル
  - 業務での重要性(3行以内)
  - 課題の詳細説明
  - 手順(番号付きリスト)
  - 完了の確認方法
  - さらに深めたい人へのキーワード(3~5個)
- 

## 4-2.「知識はあるが実務経験が浅い」向けの応用課題

あなたはIT研修の課題設計者です。

### ## 前提

研修テーマ:【例: Dockerによるコンテナ環境の構築】

対象者: Dockerの概念や基本コマンドは知っているが、  
実際のプロジェクトでの使い方は経験が少ない。

### ## 依頼

「実務に近いシナリオ形式の課題」を作成してください。

### ## シナリオ課題の仕様

架空の会社・プロジェクトの状況を設定し、

「あなたはチームに新しく参加したエンジニアです。

以下の状況に対応してください」という形式にする。

シナリオに盛り込む要素:

- 既存環境への対応(ゼロから構築ではなく)

- 何かが壊れている・設定が不足している状況
- チームメンバーへの説明・引き継ぎが必要な要素

### ## 出力形式

- シナリオ背景(200字程度)
  - あなたのタスク(箇条書き3～5個)
  - 取り組みのヒント(3個まで)
  - 想定される完成状態
  - 振り返り設問(「なぜこの方法を選んだか」など)
- 

## 4-3. 「知識を人に教える」課題(インプット→アウトプット変換)

以下の条件で「教えることで学ぶ」発展課題を作成してください。

### ## 状況

研修テーマ:【例:SQLのJOIN句の種類と使い分け】

対象者:JOIN句を使いこなせる受講者

### ## 課題の形式

「もし自分がこのテーマの講師なら」という視点で取り組む課題。

作成してほしいもの:

- 5分間のミニ解説資料(スライド構成案 or 手書きホワイトボード想定)
  - 「INNER JOIN と LEFT JOIN の違いを、データの例を使って説明する」
  - Markdown形式で構成案を作る
- 「よくある誤解TOP3」リスト



- 受講者がつまづきやすいポイントを先読みしてリストアップ

### 3. クイズ問題3問

- 基礎確認1問 / 応用1問 / 実務に近い問題1問
- 正解と解説付き

#### ## 意図

教える準備をすることで、自分の理解のあいまいな部分が明確になる。

「説明できる = 本当に理解している」を体験させる課題にしてください。

---

## 4-4. パフォーマンス・改善課題（上級者向け）

あなたは上級エンジニア向けの研修課題設計者です。

#### ## 状況

研修テーマ:【例:Pythonでのデータ処理基礎】

対象者:基礎は完璧。パフォーマンスや設計品質に興味がある。

#### ## 依頼

「動くコードをより良くする」という観点の発展課題を作成してください。

#### ## 課題の構成

以下のサンプルコード（仮。実際の研修で使ったものに置き換えてください）に対して:

```
```python
# サンプル: 1~100の偶数の合計を求める

result = 0

for i in range(1, 101):

    if i % 2 == 0:
```

```
result = result + i
```

```
print(result)
```

### タスク1:リファクタリング

同じ結果を出す、よりPythonicなコードに書き直す。少なくとも2種類の書き方を示すこと。

### タスク2:可読性の評価

元のコードと改善後のコードを比較し、「保守性・可読性・パフォーマンス」の3軸で評価表を作る。

### タスク3:エッジケースの検討

このロジックが「想定外の入力」で壊れるケースを3個挙げ、対処方法を実装する。

出力形式:Markdown。コードはコードブロックに。

---

### ## 5.【両方同時生成】差分パッケージプロンプト {#both}

演習終了後、\*\*1回のプロンプト\*\*でフォロー資料と発展課題を同時に生成します。

あなたはIT研修の教材設計者です。以下の研修について、習熟度の異なる2グループ向けの資料セットを作成してください。

## 研修情報

- テーマ:【例:Gitブランチ操作(branch / checkout / merge)】
- 今日扱った内容:
  - ブランチの概念と作成
  - チェックアウトによる切り替え
  - マージの基本(fast-forward)
  - コンフリクトの発生と解消
- 演習時間:60分

## グループA:フォローが必要な受講者

特徴:コンフリクト解消まで到達できなかった。マージの概念があいまい。

このグループへの資料:

1. ブランチを「なぜ使うのか」の説明(たとえ話付き)
2. 今日のコマンドまとめ表(コマンド・意味・使い方例)
3. コンフリクトが起きる仕組みの図解(テキストで表現)
4. 最小手順の再確認課題(ブランチ作成→コミット→マージの流れを再度やる)
5. 「次回研修までにやること」チェックリスト

## グループB: 発展を求める受講者

特徴: 演習を20分早く終えた。git rebaseやPull Requestに興味を示している。

このグループへの課題:

1. rebaseとmergeの違いを調べて比較表を作る課題
2. GitHub上でPull Requestを出す疑似ワーク(手順付きシナリオ)
3. チーム開発でのブランチ戦略(Git Flow / GitHub Flow)の調査課題
4. 「自分のチームではどちらのブランチ戦略が合うか」を考察する記述課題

## 出力形式

「グループA向け資料」「グループB向け課題」を明確に区切ること。両方ともMarkdown形式。

---

## 6. シーン別コピーペプロンプト集 {#scene}

### 研修直前(5分で準備)

研修テーマ:【テーマを入れる】今日の対象者の前提知識レベル:【入れる】懸念点:一部の受講者が【入れる】という状況になる可能性がある。

以下を5分で使える形式で作ってください:

- 理解確認用の簡単な問い(冒頭3分で使える・3問)
- レベル下の人が詰まったときの最初の一言ヒント(5個)
- レベル上の人が「早く終わったとき」に渡せる追加問い(2問)

Markdown形式、箇条書き中心で。

---

### 演習中(その場で生成)

今、研修の演習中です。

演習課題:【課題内容を入れる】詰まっているポイント:【受講者が詰まっている箇所を入れる】

30秒で口頭説明できる「最小のヒント」を3パターン作ってください。

- ヒントA:考え方の糸口だけ(答えは言わない)
- ヒントB:関係する命令・構文の名前だけ教える
- ヒントC:「最初の1行だけ書いてあげる」パターン

---

### 研修終了後(振り返り資料)

今日の研修が終わりました。

研修テーマ:【テーマ】扱った内容:【箇条書きで入れる】全体的な習熟度:【雰囲気を入れる。例:6割の受講者が演習完了。残り4割は途中まで】

以下を作成してください:

1. 今日のまとめ資料(A4 1枚分のボリューム)

- 今日学んだことの要点(5項目)
- よくあった質問とその回答(3~5個)
- 次回研修までの自習推奨事項

2. 理解確認テスト(5問)

- 基礎2問・応用2問・実務1問
- 選択肢式 or 穴埋め式
- 正解と解説付き

出力形式:Markdown

---

### サブ講師への引き継ぎメモ生成

あなたはIT研修の記録係です。

今日の研修概要:

- テーマ:【テーマ】

- 受講者数:【人数】
- 全体の進行状況:【進捗】
- 理解が遅れている受講者の傾向:【特徴】
- 理解が早い受講者の傾向:【特徴】
- 発生した主な質問・つまずき:【箇条書き】

次のサブ講師が研修を引き継ぐための「引き継ぎメモ」を作成してください。

含める内容:

- 今日カバーできた内容・できなかった内容
- 要注意受講者のタイプ(個人名なし)
- 次回研修で補足すべきポイント
- 使える追加資料・課題の提案

形式:Markdown。箇条書き中心。800字以内。

---

## ## 7. 生成物の品質を上げる追加指示集 {#quality}

基本プロンプトに追加すると出力が改善される指示です。

### ### 難易度を下げる指示

・専門用語を使う場合は必ずカッコ内に日本語説明を入れてください・中学生に説明するような平易な言葉で書いてください・コードの各行にコメントを入れてください・「なぜこれをするのか」の理由を毎ステップに添えてください・手順は最小単位まで細分化してください(「ファイルを開く」も一つの手順として書く)

### ### 難易度を上げる指示

・「なぜそうなるのか」の仕組みレベルまで説明を求めてください・複数の解法を示し、トレードオフを比較させてください・エラーが発生する前提のシナリオを含めてください・「これをチームに説明するらどう言うか」という観点を含めてください・公式ドキュメントで確認すべき箇所を示してください

### ### 出力形式を整える指示

・スライドのノート欄に貼れるよう、1スライドごとに区切ってください・印刷配布を想定し、A4縦1枚に収まる量にしてください・演習手順は「番号付きリスト」で、補足説明は「引用ブロック」で分けてください・表を多用して一覧性を高めてください・コードと説明文を交互に配置してください

---

## ## 8. PowerPoint用スライドアウトラインプロンプト {#ppt}

PPTXに変換することを前提にした、スライド構成案の生成プロンプトです。

### ### フォロー資料のスライド構成案

以下の条件でPowerPointスライドのアウトラインを作成してください。

## 資料情報

- 目的:研修についていけなかった受講者への補足資料
- テーマ:【テーマ】
- スライド枚数:8~10枚
- 配布タイミング:研修当日の終了後

## スライド構成の指定

各スライドについて以下を出力してください:

- スライド番号
- タイトル
- メインコンテンツ(箇条書き3~5行)
- ビジュアル要素の提案(図・表・アイコンのアイデア)
- 発表者ノート(口頭で補足する内容・2~3行)

## スライド構成の方針

1. 表紙(このスライドで何を得られるか明示)
2. 今日のゴールの再確認
3. 概念の図解(たとえ話で)
4. 用語まとめ表
5. 手順のおさらい(ステップバイステップ)
6. よくあるエラーと対処法
7. 練習課題(ミニマム版)
8. チェックリスト
9. 次のステップ
10. まとめ・応援メッセージ

出力形式:Markdown。スライドごとに --- で区切ること。

### ### 発展課題のスライド構成案

以下の条件でPowerPointスライドのアウトラインを作成してください。

## 資料情報

- 目的:研修を早く終えた上級者への追加課題パッケージ
- テーマ:【テーマ】
- スライド枚数:6~8枚
- 配布タイミング:演習完了後すぐ

## スライド構成

1. 表紙(「余裕があるあなたへ」というトーン)
2. 基礎の先にあるもの(今日の内容と発展の位置関係を図で)
3. 発展課題①(課題内容・目的・手順)
4. 発展課題②(課題内容・目的・手順)
5. 発展課題③(課題内容・目的・手順)
6. 調べると面白いキーワード集
7. 「学びを深めるための問い」(自己内省設問)
8. 参考になるリソースの種類(カテゴリのみ。URLは仮でよい)

各スライドの出力形式:

- スライドタイトル
- メインコンテンツ(箇条書き)
- ビジュアル提案
- 発表者ノート

Markdown形式。--- で区切ること。

---

## 9. まとめ {#summary}

本記事で紹介したプロンプトをカテゴリ別に整理します。

| カテゴリ | 用途 | 生成にかかる時間目安 |

|-----|-----|-----|

| レベル下フォロー(基本) | 演習で詰まった人への補足資料 | 1~2分 |

| 用語フォロー | 専門用語でつまづいた人向け | 1分 |

| ヒントカード | 演習中に渡す段階式ヒント | 2分 |

| 自習ロードマップ | 研修後の復習ガイド | 2～3分 |

| レベル上発展(基本) | 早く終わった人への追加課題 | 1～2分 |

| シナリオ課題 | 実務に近い応用課題 | 2～3分 |

| 教える課題 | 理解を深めるアウトプット課題 | 2分 |

| 差分パッケージ | 両グループを1回で生成 | 3～5分 |

| 研修直前5分 | 最小限の準備 | 30秒～1分 |

| 演習中即席 | その場で渡すヒント | 30秒 |

| PPTアウトライン | スライド構成案の生成 | 2～3分 |

#### ### 使い方のコツ

##### \*\*1. テーマと対象者の解像度を上げる\*\*

「プログラミング研修」より「Python基礎・演習2日目・業務未経験者対象」のほうが精度が上がります。

##### \*\*2. 生成後は必ず一読する\*\*

技術的な正確さは講師が確認してください。特にコードのサンプルはそのまま使わず、実際の研修環境で動作確認を。

##### \*\*3. テンプレートとして保存する\*\*

よく使うプロンプトは自分用にカスタマイズして保存しておく、次回からは「テーマ」「対象者」「詰まりポイント」を差し替えるだけで使えます。

##### \*\*4. 複数回生成して選ぶ\*\*

同じプロンプトを2～3回実行すると、異なる切り口の資料が出てきます。良いところを組み合わせるのが実践的です。

---

習熟度の差がある研修は、「うまくいかなかった」ではなく「対応する素材がなかった」だけです。



AIを使えば、その素材は研修当日でも10分以内に用意できます。

まず1つ、次の研修で試してみてください。

---

\*本記事は「うさうさ研修工房」が提供するIT研修講師向けコンテンツです。\*